

SatyaCheck — Backend Engineering Guide

Fake News Defense Project | Full Stack with AI

Date: July 28, 2026

Project Overview

SatyaCheck ek AI-powered web app hai jo news content analyze karke misinformation detect karta hai. Teen AI models (OpenAI + Gemini + Anthropic) ka use karke yeh ek multi-model consensus system banata hai.

Layer	Technology
Frontend	React + Vite (TypeScript)
Backend	Express 5 (Node.js, TypeScript)
Database	PostgreSQL + Drizzle ORM
AI Models	OpenAI GPT-5.2 + Gemini 2.5 Flash + Anthropic Claude
API Contract	OpenAPI spec → Orval codegen
Monorepo	pnpm workspaces

Backend Engineering — 10 Concepts Explained

1. REST API Design

Browser ya app ek **request** bhejta hai server ko, server **response** deta hai.

```
User clicks "Analyze" button
↓
Frontend sends: POST /api/analyze
  { content: "Modi ne kaha...", type: "text" }
↓
Server receives → processes → returns result
↓
Frontend shows result to user
```

Method	Kaam	Example
GET	Data maango	/api/history — purana data lao
POST	Naya data bhejo	/api/analyze — content analyze karo
PUT	Update karo	/api/user/1 — profile update
DELETE	Delete karo	/api/analysis/1 — record hatao

2. 📄 OpenAPI Contract-First Development

Code likhne se pehle API ka blueprint banao. YAML mein endpoints, request body, response shape define karna.

```
/api/analyze:
  post:
    requestBody:
      content: string
      type: text | url | image
    response:
      verdict: fake | real | uncertain
      confidence: 0-100
```

Phir **Orval tool** is YAML ko padhta hai aur automatically TypeScript code generate karta hai.

3. 📄 Zod Validation

User se aane wala data pehle **check** hota hai, phir process hota hai.

```
User ne bheja: { content: "", type: "video" }
                ↓
Zod check karta hai:
  ❌ content khali hai – error!
  ❌ "video" allowed nahi – error!
                ↓
Server returns: 400 Bad Request
```

- Schema-based validation
- Type-safe request handling
- Database mein garbage data nahi jaata

4. 📄 PostgreSQL + Drizzle ORM

PostgreSQL = ek digital almirah jisme data rows mein rakha hota hai.

```
analyses table:
| id | content          | verdict | confidence | createdAt |
|----|-----|-----|-----|-----|
| 1  | "Modi ne..."  | fake    | 87%        | 2026-07-28 |
| 2  | "NASA ne..."  | real    | 92%        | 2026-07-28 |
```

Drizzle ORM = TypeScript mein SQL likhne ka tarika

```
// Bina ORM (purana tarika)
"SELECT * FROM analyses WHERE id = 1"

// Drizzle ke saath (modern tarika)
db.select().from(analyses).where(eq(analyses.id, 1))
```

5. 🧠 AI Integration (3 Models)

```
User submits: "Vaccine se cancer hota hai"
      ↓
Step 1 – OpenAI GPT analyze karta hai:
  "Yeh claim medically false hai, confidence 94%"
      ↓
Step 2 – Gemini cross-check karta hai:
  Fact point 1: WHO data ❌ False
  Fact point 2: Medical journals ❌ False
      ↓
Step 3 – Final verdict: FAKE – 91% confidence
      ↓
User poochhe "Kyun fake hai?" → Anthropic Claude chat mein explain karta hai
```

Model	Kaam
OpenAI GPT	Primary analysis & reasoning
Google Gemini	Fact checking & verification
Anthropic Claude	Conversational explanation

6. 🛡️ Middleware (Express)

Middleware = darwaaza ke guard — request andar jaane se pehle check hota hai

```
Request aai: POST /api/analyze
      ↓
Guard 1 – CORS Check: "Kahan se aa rahi hai request?"
      ↓
Guard 2 – Body Parser: "JSON data read karo"
      ↓
Guard 3 – Rate Limiter: "100 baar request? Rok do"
      ↓
Route Handler: Actual kaam hota hai
      ↓
Response wapas jaati hai
```

7. 📝 Pino Logging

```
// console.log – amateur tarika
console.log("Request aai")

// Pino – professional tarika
req.log.info({
  method: "POST",
  url: "/api/analyze",
  responseTime: "234ms",
  status: 200
})
```

Production mein debug kar sakte ho — errors, slow requests, user activity sab track hota hai.

8. 🔑 Environment Variables & Secrets

```
# .env file (GitHub pe nahi jaati)
OPENAI_API_KEY=sk-abc123...
GEMINI_API_KEY=AIza...
DATABASE_URL=postgresql://...
```

```
// Code mein sirf variable name
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
})
```

Kyun zaroori: Key code mein hogi toh koi bhi use kar sakta hai — aapka OpenAI bill lakhon ka aa sakta hai! 🤖

9. 📄 Deployment — render.yaml

```
services:
  - type: web
    name: satyacheck
    buildCommand: pnpm install && pnpm build
    startCommand: node server.js
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: satyacheck-db

databases:
  - name: satyacheck-db
    plan: free
```

Render yeh file padhta hai aur automatically server + database banata hai aur live URL deta hai. **Infrastructure as Code** kehte hain ise.

10. 📄 Monorepo — pnpm Workspaces

```
workspace/
├── lib/
│   ├── api-spec/      ← OpenAPI spec (ek baar likho)
│   ├── api-client/    ← Generated hooks (sabko milta hai)
│   ├── db/            ← Database code (sabko milta hai)
│   └── ai-clients/    ← AI wrappers (sabko milta hai)
├── artifacts/
│   ├── frontend/      ← React app (lib se import karo)
│   └── api-server/    ← Express app (lib se import karo)
```

Ek jagah type change karo → everywhere update. Google, Meta, Microsoft sab yahi use karte hain.

📄 Complete Request Flow Diagram

```
User Request
  ↓
Middleware (CORS → Body Parse → Rate Limit → Log)
  ↓
Route Handler (/api/analyze)
  ↓
Zod Validation → ❌ Error? → 400 Response
  ↓ ✅ Valid
AI Models (OpenAI + Gemini parallel)
  ↓
Results Combined → Confidence Score
  ↓
Database Save (Drizzle → PostgreSQL)
  ↓
JSON Response → Frontend
  ↓
User Result Dikhta Hai 🎉
```

📌 4 Students Work Division — Start to End

Phase	Task	Student 1 (Backend)	Student 2 (Frontend)	Student 3 (AI/ML)	Student 4 (DB+DevOps)
1 Planning	Project planning	API endpoints list	Pages & UI flow	AI models research	Database tables plan
2 Contract	OpenAPI Spec	📄 Endpoints define	Review data shapes	AI response format	DB schema fields
3 Codegen	Orval run	Command chalao	Hooks check karo	Zod schemas verify	—
4 Frontend	React pages	API integration	📄 Sare pages banao	AI response UI	—
5 Backend	Express routes	📄 All /api routes	Frontend se test	AI logic in routes	DB calls add
6 AI	3 AI models	Routes ready rakho	Loading/error UI	📄 OpenAI+Gemini+Claude	AI results save
7 Database	Schema+migration	DB calls in routes	History/dashboard	AI results format	📄 Drizzle schema
8 Testing	End-to-end test	Postman API test	Manual page test	AI accuracy check	DB queries check
9 Deploy	GitHub + Render	render.yaml config	Vite build optimize	AI keys production	📄 PostgreSQL setup
10 Docs	README + Slides	API docs	Screenshots + UI	AI pipeline explain	📄 README + diagram

📌 Backend Engineering Summary

Concept	Real World Use
REST API Design	Har company ka backend isi pe chalta hai
OpenAPI Contract	Industry standard documentation
Zod Validation	Security aur data quality
PostgreSQL + ORM	Permanent data storage
AI Integration	Modern apps ka core feature
Middleware	Authentication, logging, security
Environment Variables	Industry standard secret management
Logging (Pino)	Production debugging & monitoring
Deployment (render.yaml)	Actual users tak app pahunchana
Monorepo (pnpm)	Google, Meta jaisi companies ka approach

📌 *Is project se tumne ek **production-grade backend** banana seekha — sirf tutorial nahi, real engineering!*